

Claude Code 플러그인 활용 가이드

GD RDM 프로젝트 기준 추천 플러그인 및 설치/설정 정리

작성일: 2026년 6월 21일

기본 설치 절차

공식 마켓플레이스(claude-plugins-official)는 Claude Code 실행 시 자동으로 사용 가능합니다. 별도로 추가할 필요 없이 바로 설치만 하면 됩니다.

```
# 설치
/plugin install <플러그인이름>@claude-plugins-official

# 사용 가능한 플러그인 둘러보기
/plugin
# → Discover 탭에서 탐색, 또는 claude.com/plugins 에서 브라우저로 확인
```

설치가 안 될 경우: "plugin not found" 메시지가 뜨면 마켓플레이스가 없거나 오래된 것이므로 아래 명령으로 해결합니다.

```
/plugin marketplace update claude-plugins-official
# 처음이라면
/plugin marketplace add anthropics/claude-plugins-official
```

우선순위 추천

1순위 — CLAUDE.md Management, Pyright LSP, Commit Commands (당장 워크플로우에 바로 꽃힘)

2순위 — Data, Feature Dev (대시보드 기능 본격적으로 만들 때)

3순위 — Code Review, Security Guidance (내부 배포 전 검수 단계)

항목별 상세 설명

1. CLAUDE.md Management

설치 명령어

```
/plugin install claude-md-management@claude-plugins-official
```

사용 예

rdm_analysis.py에서 새로운 컬럼 처리 로직을 추가하고 세션을 끝낼 때, /claude-md-management:audit 명령으로 현재 CLAUDE.md가 최신 상태인지, 누락된 컨텍스트가 있는지 점검합니다. 예를 들어 "542개 컬럼 중 37개는 categorical, 나머지는 numeric" 같은 사실이 코드에서만 확인되고 CLAUDE.md엔 적혀있지 않다면, 이 플러그인이 다음 세션에서 같은 질문이 반복되지 않도록 캡처해줍니다.

왜 유용한가

Schema-First 접근에서 가장 중요한 것은 메타데이터를 정확히 기억하는 것인데, 사람이 매번 CLAUDE.md를 손으로 업데이트하면 누락이 생깁니다. 이 플러그인이 그 갭을 메웁니다.

세부 설정

설치 후 별도 설정 없이 바로 슬래시 명령 사용 가능합니다.

```
/claude-md-management:audit # 현재 CLAUDE.md 품질 점검
```

프로젝트 루트(CLAUDE.md가 있는 디렉토리)에서 실행하세요.

2. Pyright LSP

설치 명령어

```
/plugin install pyright-lsp@claude-plugins-official
```

사용 예

rdm_merge.py에서 여러 CSV를 병합하는 함수를 작성할 때, pandas DataFrame의 컬럼 타입이 함수 시그니처와 안 맞으면 코드를 쓰는 즉시(실행 전에) 타입 불일치를 잡아냅니다. 대용량 CSV(2GB급) 작업에서 타입 에러로 런타임에 실패하면 디버깅 비용이 크기 때문에, 사전에 걸러지는 게 시간을 많이 아껴줍니다.

왜 유용한가

Schema-First 패턴 특성상 Claude는 실제 데이터를 직접 보지 못하고 메타데이터만

보고 코드를 생성합니다. 그러다 보니 타입 추론이 빛나갈 여지가 일반 코딩보다 큼니다 — LSP가 이 리스크를 줄여줍니다.

세부 설정

설정 불필요 — 설치만 하면 .py 파일 편집 시 자동으로 백그라운드에서 타입 진단을 돕습니다.

별도 pyrightconfig.json을 프로젝트 루트에 두면 더 세밀하게 제어 가능합니다.

```
{
  "include": ["rdm_*.py"],
  "typeCheckingMode": "basic"
}
```

3. Commit Commands

설치 명령어

```
/plugin install commit-commands@claude-plugins-official
```

사용 예

rdm_data_generator.py에 synthetic data 생성 로직을 추가한 뒤 /commit-commands:commit을 실행하면, diff를 분석해서 컨벤션에 맞는 커밋 메시지를 자동 작성합니다 (예: "feat(generator): add categorical column synthesis for GD schema").

왜 유용한가

네 개 기능(generator/merge/analysis/dashboard)을 오가며 작업하다 보면 커밋 메시지가 점점 부실해지기 쉬운데, 이 플러그인이 일관성을 유지시켜줍니다.

세부 설정

```
/commit-commands:commit # diff 분석 후 커밋 메시지 자동 생성
```

팀 컨벤션이 있다면 CLAUDE.md에 "커밋 메시지는 conventional commits 형식 사용" 같은 규칙을 적어두면 이를 참고합니다.

4. Code Review

설치 명령어

```
/plugin install code-review@claude-plugins-official
```

사용 예

웹 대시보드 기능을 다 만들고 내부망 배포 전에 PR을 올리면, /code-review:review가 보안 이슈(SQL 인젝션, 입력값 검증 누락), 로직 버그, 컨벤션 위반을 신뢰도 순으로 정리해서 보여줍니다. "이건 99% 확실한 버그" vs "이건 스타일 제안" 식으로 구분돼서, 노이즈에 묻히지 않습니다.

왜 유용한가

내부 배포 전 최종 검수 단계로 쓰기 좋습니다 — 로컬에서 도는 리뷰이기 때문에 정책 충돌 가능성이 낮은 편입니다(다만 설치 전 확인은 필요).

세부 설정

```
/code-review:review      # 현재 변경사항 또는 PR 리뷰
```

신뢰도 기반 필터링은 별도 설정 없이 기본 작동하며, 특정 PR을 지정하려면 PR 번호나 브랜치명을 함께 입력하면 됩니다.

5. Security Guidance

설치 명령어

```
/plugin install security-guidance@claude-plugins-official
```

사용 예

웹 대시보드에서 사용자 입력(필터 조건 등)을 받아 동적 쿼리를 만드는 코드를 작성할 때, 파일을 저장하는 시점에 자동으로 "이 패턴은 SQL 인젝션에 취약할 수 있음" 경고가 뜹니다. 작성 후가 아니라 작성하는 순간 잡힙니다.

왜 유용한가

사내 보안 규정이 엄격한 프로젝트라면, 코드 리뷰 단계가 아니라 작성 단계에서 이미 걸러지는 게 안전합니다.

세부 설정

설치하면 hook으로 동작 — 파일 저장 시 자동 트리거됩니다.

hooks는 설치 후 /reload-plugins 또는 재시작이 필요합니다. 경고 민감도 조정이

필요하면 .claude/settings.json에서 plugin별 옵션을 확인하세요.

6. Feature Dev

설치 명령어

```
/plugin install feature-dev@claude-plugins-official
```

사용 예

웹 대시보드에 새 기능(예: 결함 트렌드 시각화)을 추가할 때, 막연히 "대시보드에 트렌드 차트 추가해줘"라고 하면 (1) 기존 코드베이스 구조 탐색, (2) 어떤 차트 라이브러리/패턴을 쓸지 설계안 제시, (3) 구현, (4) 자체 리뷰까지 순서대로 진행합니다.

왜 유용한가

4개 기능이 서로 연결된 프로젝트에서 기존 구조와 일관성 있게 만들어지도록 강제합니다.

세부 설정

```
/feature-dev:start "대시보드에 결함 트렌드 차트 추가"
```

탐색-설계-구현-리뷰 단계를 순차로 밟으며, 중간에 설계안에 대해 직접 피드백을 주고받을 수 있습니다.

7. Data (Anthropic 공식)

설치 명령어

```
/plugin install data@claude-plugins-official
```

사용 예

synthetic data로 분석 로직을 검증할 때, "542개 컬럼 중 결측치 비율이 30% 넘는 컬럼 찾아서 막대그래프로 보여줘" 같은 요청을 하면, 코드 생성→실행→시각화까지 한 흐름으로 처리합니다.

왜 유용한가

Schema-First 패턴의 "로컬 실행" 부분과 직접 맞닿아 있습니다 — 메타데이터 기반으로 작성된 코드를 데이터 분석 워크플로우에 맞게 다듬어줍니다.

세부 설정

```
/data:explore    # 데이터셋 탐색  
/data:query      # SQL 작성/실행
```

로컬 CSV나 DB 연결이 필요하다면 작업 디렉토리에 데이터 파일 경로를 명시해서 요청하면 됩니다.

8. Frontend Design

설치 명령어

```
/plugin install frontend-design@claude-plugins-official
```

사용 예

웹 대시보드의 첫 레이아웃을 잡을 때, 일반적인 부트스트랩 느낌 기본 UI 대신 결합 진단 데이터 특성에 맞는 타이포그래피·색상·레이아웃을 제안받을 수 있습니다.

왜 유용한가

사내 배포용 대시보드라면 "누가 봐도 AI가 만든 템플릿"이라는 인상을 주지 않는 게 중요할 수 있습니다.

세부 설정

별도 명령어보다는, 대화에서 "대시보드 UI를 만들어줘"라고 요청하면 자동으로 트리거됩니다.

설정은 거의 없으며, 디자인 방향성(색상 팔레트, 톤)을 프롬프트에 구체적으로 적을수록 결과가 좋습니다.

9. Session Report

설치 명령어

```
/plugin install session-report@claude-plugins-official
```

사용 예

한 주간 작업한 세션들을 모아서 리포트를 뽑으면, 어떤 작업에 토큰을 많이 썼는지(예: 대규모 CSV 스키마 분석 vs 단순 버그픽스) 파악할 수 있습니다.

왜 유용한가

Schema-First 패턴을 쓰는 이유 자체가 토큰 절약인데, 실제로 어디서 토큰이 새는지

가시화해줍니다.

세부 설정

```
/session-report:generate
```

실행하면 토큰/캐시 효율, 서브에이전트.스킬 사용 현황을 HTML로 뽑아줍니다. 별도 설정 없이 바로 작동합니다.

프로젝트 전체 설정 팁

여러 명이 함께 프로젝트를 진행한다면, 마켓플레이스와 설치 플러그인 목록을 .claude/settings.json에 커밋해두면 팀 전체가 클론 시 동일한 플러그인 카탈로그를 자동으로 받게 됩니다.

```
/plugin marketplace list    # 현재 등록된 마켓플레이스 확인
```

참고 사항

- 위 플러그인들은 대부분 로컬에서 동작하지만(LSP, 코드 리뷰 혹 등), 설치 전 사내 보안팀에 plugin 이 외부로 어떤 데이터를 전송하는지(특히 Data, Feature Dev 처럼 분석 작업을 수행하는 것들) 확인하는 것을 권장합니다.
- 각 플러그인 상세 페이지(claude.com/plugins/<이름>)에서 소스코드 링크를 확인할 수 있으니, 설치 전 검토를 권장합니다.
- 개인 프로젝트(주식 모니터링, 네이버 카페 다운로더)는 외부 API(KIS, 카카오) 연동이라 공식 MCP 보다는 직접 Python 스크립트 + cron/launchd 조합이 더 적합합니다.